IAC TECHNOLOGY DIVISION

TITLE:

MUSOLSONG COORDINATOR SOFTWARE SYSTEM - INTRODUCTORY GUIDE

PROJECT / DESTINATION:

MUSOL - ESPECTRO POLARÍMETRO SOLAR

Document Code: DS/TN-MUS/0011v1.1

Issue: 1.0
Date: 24-02-2026
External code: N/A
PBS code: N/A
Work: N/A
Status: N/A

INSTITUTO DE ASTROFISICA DE CANARIAS
38200 La Laguna (Tenerife) - ESPAÑA - Phone (922)605200 - Fax (922)605210

AUTHORS LIST

Name	Institution	Function	Date	Signature
Heidy Moreno Arce	IAC		24-02-2026	

APPROVAL CONTROL

	Name	Institution	Function	Date	Signature
Revised				24-02-2026	
Approved				24-02-2026	
Authorized				24-02-2026	

DOCUMENT CHANGES RECORD

Issue	Date	Change Description
1.1	24-02-2026	Modify script ***add_route_to_plc_and_check_connection_in_linux.py***
1.0	05-02-2026	Initial Issue

SUMMARY

This document describes briefly the MUSOLSONG software system. A software system designed to coordinate calibration and observation procedures between the MUSOL polarimeter and SONG spectrograph.

LIST OF ABBREVIATIONS

Abbreviation	Description
AD	Applicable Document
IAC	Instituto de Astrofísica de Canarias
RD	Referenced Document
TBC	To Be Confirmed
TBD	To Be Defined

TABLE OF CONTENTS

AUTHORS LIST **2**

APPROVAL CONTROL **2**

DOCUMENT CHANGES RECORD **2**

SUMMARY **3**

THIS DOCUMENT DESCRIBES BRIEFLY THE MUSOLSONG SOFTWARE SYSTEM. A SOFTWARE SYSTEM DESIGNED TO COORDINATE CALIBRATION AND OBSERVATION PROCEDURES BETWEEN THE MUSOL POLARIMETER AND SONG SPECTROGRAPH… **3**

LIST OF ABBREVIATIONS **4**

1. INTRODUCTION **7**

1.1. BACKGROUND 7

2. OVERVIEW **7**

2.1. KEY COMPONENTS 7

2.2. SYSTEM REQUIREMENTS 7

2.2.1. Hardware Requirements 7

2.2.2. Software Requirements 7

2.2.3. Operating System 8

2.3. SOURCE CODE DIRECTORY STRUCTURE 8

2.4. INSTALLATION FOR DEVELOPMENT 9

2.5. INSTALLATION FOR END USERS (FROM A WHEEL FILE) 9

2.6. TESTING THE INSTALLATION 10

2.6.1. Configuration File 10

2.6.2. LAUNCHING THE APPLICATION 10

2.6.3. Initial System Check 11

2.6.4. GUI Interface Status Indicators 11

2.7. GUI INTERFACE OVERVIEW: MAIN COMPONENTS 11

2.7.1. Menu Options 11

2.7.2. Project Information 11

2.7.3. Operations Tab 11

2.7.4. Engineering Tab 13

2.8. CLI INTERFACE: 14

2.8.1. Usage: 14

2.8.2. Examples 15

2.9. KEY FEATURES 15

2.9.1. Sequence Management 15

2.9.2. Automated Processing 15

2.9.3. Manual Control....15
2.10. CONFIGURATION FILES....15
2.10.1. System Configuration (systemConfig.yaml)....15
2.10.2. Sequence Files (YAML)....16
2.11. TROUBLESHOOTING....16
2.11.1. Common Issues....16
2.11.2. Log Files....19
2.11.3. Support....19

1. INTRODUCTION

1.1. Background

MUSOLSONG is part of a research project aimed at upgrading the Solar-SONG instrument, which operates on the Hertzsprung-SONG telescope at the Teide Observatory. The main objective of this project is to study the Sun's global and cyclical magnetism, complemented by investigations of its interior to better understand the origin of our star's cyclical activity.

The project involves the design, manufacture, and integration of the MUSOL polarimeter, its coupling with the SONG spectrograph, and the provision of all the necessary infrastructure to ensure the proper operation of the polarimeter.

2. OVERVIEW

MUSOLSONG is a software project containing two software applications: The ***musolsong-controller*** and the ***musolsong-yamleditor.***

This document is an introductory guide to present the musolsong-controller software. This system has been designed to coordinate calibration and observation procedures between the MUSOL polarimeter and SONG spectrograph. This Python-based application provides a graphical interface and a command line interface for managing operational sequences defined via YAML configuration files, enabling automated control of spectro-polarimeter operations.

2.1. Key Components

- **musolsong-controller**: Main application for coordinating operations (GUI and CLI)
- **musolsong-yamleditor**: GUI application for creating/editing YAML configuration files
- **Polarimeter Control**: Interface to MUSOL polarimeter (external python library "musol")
- **Spectrograph Control**: XML-RPC client for SONG spectrograph
- **Simulation Modules**: Simulators for testing without hardware

2.2. System Requirements

2.2.1. Hardware Requirements

- MUSOL polarimeter and PLC controller (or simulator for testing)
- SONG spectrograph system (or simulator for testing)
- Network connectivity between systems (For the real environment, all must be on the same network)

2.2.2. Software Requirements

- **Python**: Version 3.7 or higher
- **PyQt6**: Version 6.9.1 or higher. Used for graphical user interface

- **PyYAML**: Version 5.4.1 . Used for system configuration and YAML sequences files handling
- **Additional Dependencies**: As required by musol library and XML-RPC SONG client
  - For the XML-RPC SONG client there is no special requirement since Python has **built-in support** for XML-RPC.
  - The `musol library` requires the `pyads` package. Since `musol` is a standalone package, its dependencies should be resolved within it

2.2.3. Operating System

- Linux (primary development environment)
- Should be compatible with other systems supporting Python and PyQt6 (Tested on Windows)

2.3. Source code directory Structure

```
MUSOLSONG/  (project root)
├── musolsong/ (top-level Python package holding 2 apps: controller and yamleditor)
│   ├── __init__.py
│   ├── controller
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── etc/
│   │   │   └── systemConfig.yaml
│   │   ├── cli/
│   │   │   ├── __init__.py
│   │   │   ├── sequence_processor.py
│   │   ├── gui/
│   │   │   ├── __init__.py
│   │   │   ├── background_QT_processing.py
│   │   │   ├── main_Qt_window.py
│   │   ├── logging_module
│   │   │   ├── __pycache__
│   │   │   └── system_logger.py
│   │   ├── musol_lib_simulator/
│   │   │   ├── __init__.py
│   │   │   ├── musol_simulator.py
│   │   ├── musolsong_tools/
│   │   │   ├── __init__.py
```

```
│   │   │   ├── yaml_sequence_validator.py
│   │   │   └── yaml_system_config_reader.py
│   │   ├── song_client_module/
│   │   │   ├── __init__.py
│   │   │   └── song_xmlrp_client.py
│   │   └── song_server_simulator/
│   │       ├── __init__.py
│   │       ├── main.py
│   │       ├── song_server.log
│   │       └── xmlrpc_song_server_simulator.py
│   └── yamleditor
│       ├── __init__.py
│       ├── main.py
│       └── yaml_editor_gui/
│           └── yaml_sequences_editor_gui.py
├── pyproject.toml
└── README.md
```

2.4. Installation for development

The `musolsong-controller` application is installed as part of the musolsong package.

The following instructions assume that the user has its own local python and a valid musolsong distribution file, such as : *musolsong-2025.0.1.0.tar.gz*

- Just extract the data from the tar file
- On the project root directory should be a file pyproject.toml
  - Build a distribution
    ```
    python -m build
    ```
  - Install in development mode.
    ```
    pip install -e .
    ```

The project is in : https://gitiac.ll.iac.es/hmoreno/musolsong

```
pip install git+https://gitiac.ll.iac.es/hmoreno/musolsong.git
```

2.5. Installation for end users (from a wheel file)

- Copy the *musolsong-2025.1.0.0-py3-none-any.whl* file to your project root directory
- type:
  - `pip install musolsong-2025.1.0.0-py3-none-any.whl`

2.6. Testing the Installation

2.6.1. Configuration File

There is a **`systemConfig.yaml`** file that comes included in the musolsong package:

Location for developers installation:

`musolsong/controller/etc/systemConfig.yaml`

Location for end users

`$HOME/.local/lib/python3.10/site-packages/musolsong/controller/etc/systemConfig.yaml`

It contains::

plc_MUSOL:
ams_net_id: "192.168.1.100.1.1" # PLC AMS Net ID of the polarimeter
logs_enabled: true
logs_path: "/$HOME/logs" # logs directory absolute path. The directory must exist

spectrograph_SONG:
host: "127.0.0.1" # IP address of the SONG server (default 127.0.0.1)
port: 8050 # Port number of the SONG server
timeout: 30 # Timeout in seconds

MUSOLSONG_system:
logs_enabled: true # Set to true or false
logs_path: "NONE" # Logs directory absolute path, if do not want a log file, state NONE
logs_level: "DEBUG" # Log level
logs_console_enabled: true # Set to true or false

2.6.2. Launching the Application

For developers: Run the main application from the command line:

```
cd <project_dir>/musolsong/controller
```

For the GUI app:

```
python main.py
```

For the CLI:

```
python main.py--sequence-yaml config.yaml --project-name "My Project" --project-number 123
```

For end users: Once the musolsong package has been installed:

GUI app:

```
musolsong-controller
```

CLI app:

```
musolsong-controller --sequence-yaml config.yaml --project-name "My Project" --project-number 123
```

The application can be run in either real mode or simulation mode.

To run in **real mode**, simply start the application with:

```
musolsong-controller
```

To run in simulation mode, first launch the SONG simulator software.

The SONG server simulator must be started in a separate terminal using:

```
musolsong-songsimulator
```

Once the simulator is up and running, start the controller by typing:

```
USE_MUSOLLIB_SIMULATOR=1 musolsong-controller
```

2.6.3. Initial System Check

Upon startup, the system will:

- Load configuration from `systemConfig.yaml`
- Initialize the system logger
- Attempt connection to MUSOL PLC
- Attempt connection to SONG spectrograph
- Display connection status in the status bar if GUI mode

2.6.4. GUI Interface Status Indicators

- **Green Status Bar**: Both PLC and SONG server connected and ready
- **Red Status Bar**: One or both systems not available
- Only the Engineering tab is available if connections of one of the systems fail

2.7. GUI Interface Overview: Main Components

2.7.1. Menu Options

- **File → Load sequence YAML file**: Load predefined observation/calibration sequences
- **File → Exit**: Close the application

2.7.2. Project Information

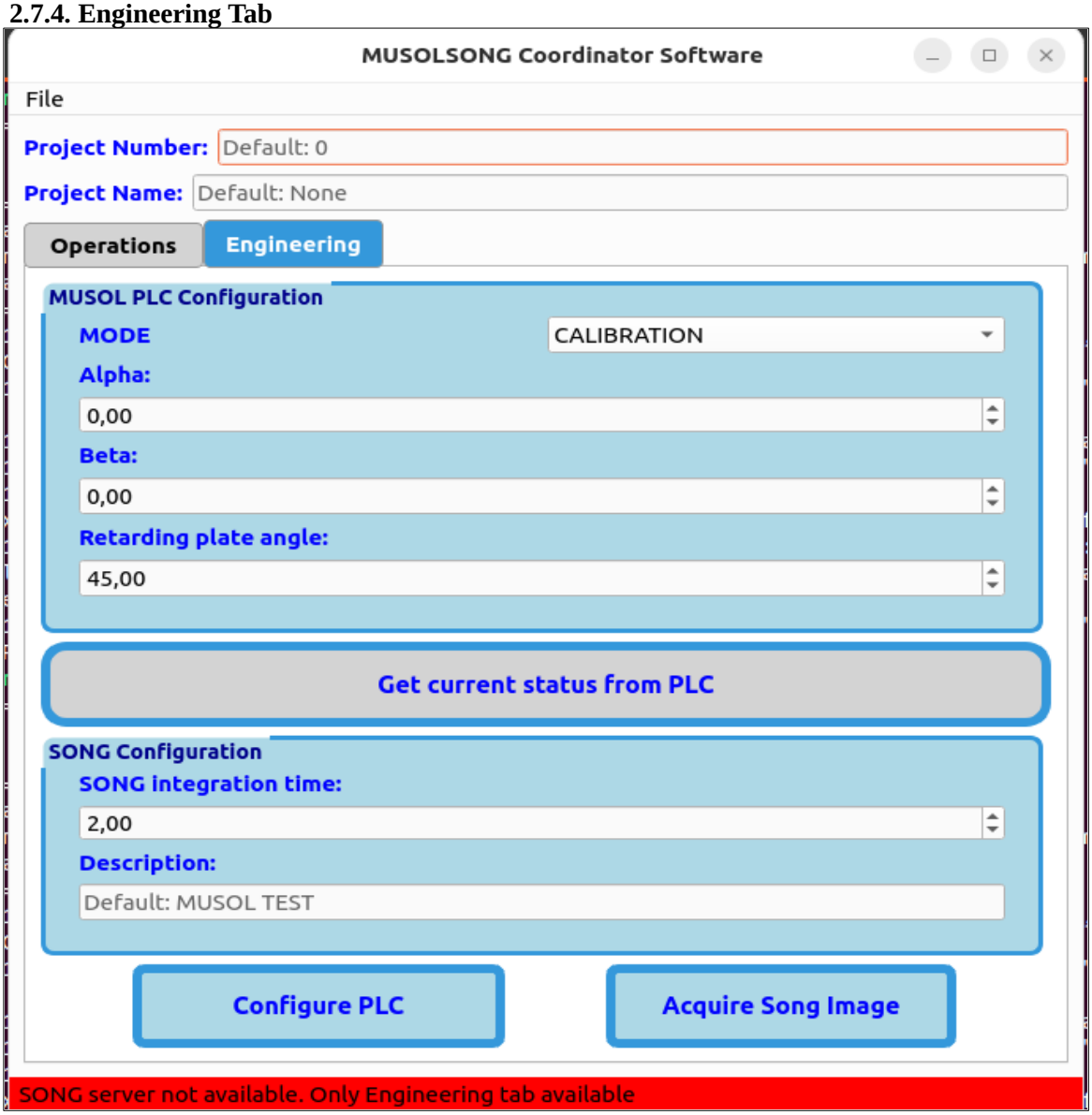
- **Project Number**: Numerical identifier (default: 0) as defined in the soda.phys.au.dk by the SONG Sci. team
- **Project Name**: Descriptive name for the observation session as defined in the soda.phys.au.dk by the SONG Sci. Team

2.7.3. Operations Tab

Used for automated sequence processing:

- **Mode Display**: Shows current operational mode (Observation/Calibration)
- **Modulations Table**: Displays predefined loaded observation/calibration sequence steps with Alpha, Beta angles, integration time, and descriptions
- **Calibration Setup**: Configuration for calibration cycles and incremental angles
- **Progress Tracking**: Real-time progress bar and status updates
- **Control Buttons**: START and ABORT processing buttons
- **Current Status Panel**: Live display of system parameters during operation

2.7.4. Engineering Tab



Used for **manual control** and testing:

- **MUSOL PLC Configuration**:
  - Operational mode selection (CALIBRATION/OBSERVATION)
  - Alpha and Beta angle controls
  - Retarding plate angle configuration (CALIBRATION mode only)
  - Button to send the configuration to PLC
- **PLC Current Status**:
  - Button to request the current operational mode and motor positions.
  - Received status information is shown on the current Status Panel
- **SONG Configuration**:

- Integration time settings
- Acquisition description
- Button to send image acquisition to SONG spectrograph server

2.8. CLI Interface:

The CLI interface can be used to process a sequence modulation file or to simply validate a file if the `--validate-only` flag is selected. Traces are shown on the standard output if verbose mode is enabled.

2.8.1. Usage:

musolsong-controller [--sequence-yaml SEQUENCE_YAML] [--project-name PROJECT_NAME] [--project-number PROJECT_NUMBER] [--validate-only] [--verbose] [--help]

Options:

General

Option	Alias	Description
--help	-h	Show this help message and exit

CLI Mode Arguments

Option	Alias	Description
--sequence-yaml SEQUENCE_YAML		Path to sequence YAML modulations file (enables CLI mode) |
--project-name PROJECT_NAME	-n PROJECT_NAME	Project name (required for CLI mode)
--project-number PROJECT_NUMBER	-p PROJECT_NUMBER	Project number (required for CLI mode)
-validate-only		Only validate the YAML file without processing
--verbose	-v	Enable verbose output

2.8.2. Examples

GUI Mode

musolsong-controller

CLI Mode

musolsong-controller --sequence-yaml config.yaml --project-name "My Project" --project-number 123

CLI Validate Only

musolsong-controller --sequence-yaml config.yaml -n "Test" -p 456 --validate-only

2.9. Key Features

2.9.1. Sequence Management

- Load YAML files defining observation/calibration sequences
- Validate sequence configurations before execution
- Visual representation of modulation steps

2.9.2. Automated Processing

- Execute complete observation sequences automatically
- Real-time progress monitoring
- Safe abort capability during execution

2.9.3. Manual Control

- Direct control of MUSOL polarimeter parameters
- Manual SONG spectrograph image acquisition
- Engineering controls for testing and troubleshooting

2.9.4. Simulation Mode

- Built-in simulators for MUSOL library and SONG server
- Test functionality without hardware connections
- Development and training capabilities

2.10. Configuration Files

2.10.1. System Configuration (`systemConfig.yaml`)

- Connection parameters for hardware components (PLC and SONG IPs and Ports)
- Logging settings and paths
- System behavior configurations

- See section 2.5.2 for a file example

2.10.2. Sequence Files (YAML)

Define observation sequences with:

- Operational mode (observation/calibration)
- Modulation parameters (alpha, beta angles)
- Integration times
- Descriptions for each step
- Calibration-specific settings (offset angles, cycles)

2.11. Troubleshooting

2.11.1. Common Issues

Connection Failures:

- Ensure that all hardware systems are powered on and accessible.
- Check AMS Net ID and IP/port settings in the configuration file.
- Verify network connectivity to the MUSOL PLC and SONG server.

MUSOL-PLC Connection on Linux

When running the application on Linux, there might be more issues to be resolved. Also check:

- The ADS port 48898 for remote connections is open.

  Command:
  nc -zv MUSOL-PLC 48898
- PLC configuration: Ensure the PLC is running in the right mode (not configuration mode) and listens for TCP connections on the expected port.
- Firewall and port filtering: Even if a TCP port is open on Linux, check there is no firewall on the PLC side blocking your connection or filtering by IP/AMS net ID.
- The firewall on your Linux machine must allow incoming connections from MUSOL-PLC using ports in the range: 32768 – 60999.
- Routing configuration: Beckhoff AMS communication requires proper routing setup between the Linux PC and the PLC. you may need to manually add the route to the PLC's AMS net ID and IP address. Without a proper route, the connection times out even if ping works. You can add and verify route from Linux to PLC with a command or tool equivalent to pyads.add_route.

To test the Linux connection, you can use the following python script which adds the route, connects to the MUSOL-PLC and attempts to read from the PLC

add_route_to_plc_and_check_connection_in_linux.py

```
import pyads
import platform
import socket

# create some constants for connection
TARGET_AMS_ID = "5.110.208.2.1.1"  # PLC AMS NetID
```

```
TARGET_NAME = "musol-plc"
TARGET_USERNAME = "Administrator"
TARGET_PASSWORD = "1"
ROUTE_NAME = "musolsong"

# Check if the script is running on Linux
if platform.system() == "Linux":
    print("\n*************************************************")
    print("\tRunning on Linux")
    # Get Target_IP
    target_IP = socket.gethostbyname(TARGET_NAME)
    print("\tTARGET_NAME:", TARGET_NAME)
    print("\tTARGET_IP:", target_IP)
    # Get the hostname and IP address
    hostname = socket.gethostname()
    print("\tHostname:", hostname)
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 80))  # Google's public DNS server
    client_IP = s.getsockname()[0]
    print("\tLocal IP address:", client_IP)
    client_NETID = ".".join(client_IP.split(".")[0:4] + ["1.1"])
    print("\tLocal NETID:", client_NETID)
    print("*************************************************\n")
    s.close()
else:
    print("Running on Windows - This script is only for Linux")
    exit

# Open the ADS port
pyads.open_port()
"""
Add a new route to the target PLC

The route is added automatically to client on Linux,
but on Windows use the TwinCAT router to add the route.
"""
print("***Add new route to PLC - add_route_to_plc......\n")
print(f"*****Route Name: {ROUTE_NAME}")
print(f"*****Client NETID: {client_NETID}")
print(f"*****Client IP: {client_IP}")
print(f"*****Target IP: {target_IP}")
print(f"*****Target AMS NetID: {TARGET_AMS_ID}")
print(f"*****Target Username: {TARGET_USERNAME}\n")
retval =pyads.add_route_to_plc(
    client_NETID, client_IP, target_IP, TARGET_USERNAME, TARGET_PASSWORD,
    route_name=ROUTE_NAME
)
print(f"***Route added to PLC - return value from add_route_to_plc: {retval}\n")
print("*************************************************\n")
# Close the ADS port
pyads.close_port()

"""
Try to connect to the PLC using TwinCAT3.

The connection is established using the TARGET_AMS_ID, TARGET_IP,
and the TwinCAT3 port number (851).

Raises:
    Exception: If the connection to the PLC cannot be established.
"""
try:
    print ("***Connect to plc......\n")
    print (f"***TwinCAT3 AMS ID: {TARGET_AMS_ID}")
    print (f"***TwinCAT3 IP: {target_IP}")
    print (f"***TwinCAT3 port number: {pyads.PORT_TC3PLC1}\n")
    #pyads.PORT_TC3PLC1 is por 851
    plc = pyads.Connection(
        TARGET_AMS_ID, pyads.PORT_TC3PLC1, target_IP
    )
    print(f"***Successful connected to plc\n")
    print("*************************************************\n")


except Exception as e:
    print(f"Failed to connect: {e}")


try:
    plc.open()
except Exception as e:
    print(f"Failed to open connection: {e}")
    exit()
```

```
print(f"***Successful open connection to plc\n")

print("**************************************************\n")
print ("***Check PLC-Read State -  plc.read_state" )

try:
    """
    Read the state of the PLC.

    The state of the PLC is read from the PLC using the read_state method.
    The state is returned as a retval object.

    Raises:
        Exception: If the read_state method fails.
    """
    retval = plc.read_state()
    print(f" return value from read_state: {retval} ")

except Exception as e:
    """
    Failed to read the state of the PLC.

    The exception is raised if the read_state method fails.
    """
    print(f"Failed to read_state: {e}")

print("**************************************************\n")
print ("***Check PLC-Read .bEmergencyStop - plc.read_by_name" )
# read a well known bool value by name
try:
    i = plc.read_by_name(".bEmergencyStop", pyads.PLCTYPE_BOOL)
    print (f"return value from read_by_name .bEmergencyStop:  {i}")
except Exception as e:
    print(f"Failed to read_state: .bEmergencyStop: {e}")

print("**************************************************\n")

"""
Try to read the CPU load from the PLC.

The CPU load is read from the PLC using the read_by_name method,
with the name "_SystemInfo.CpuLoad" and the data type pyads.PLCTYPE_REAL.

Raises:
    Exception: If the connection to the PLC cannot be established or if the
    read_by_name method fails.
"""
print ("***Check PLC-Read _SystemInfo.CpuLoad - plc.read_by_name" )
try:
    # Read the CPU load from the PLC
    cpu_load = plc.read_by_name("_SystemInfo.CpuLoad", pyads.PLCTYPE_REAL)
    print(f"CPU load: {cpu_load:.2f}%")
except Exception as e:
    """
    Failed to read the CPU load from the PLC.

    The exception is raised if the read_by_name method fails.
    """
    print(f"***Failed to read_by_name _SystemInfo.CpuLoad: {e}")
print("**************************************************\n")

# close connection
try:
    plc.close()
except pyads.ADSError as e:
    print(f"ADS/AMS connection failed: {e}")
```

Error Indications

An error such as:

Failed to read_state: ADSError: timeout elapsed (1861).

might indicate a firewall issue on your system, specifically blocking ADS/AMS incoming communication. If firewall is accepting connections from PLC, it might indicate a problem on the ADS/

AMS PLC connections. As per today (24/02/2026), this issue is not resolved and requires a reset of the PLC.

YAML File Loading Errors:

- Validate YAML syntax using standard validators
- Check sequence file against expected schema

Processing Failures:

- Verify both systems are connected before starting sequences
- Check log files for detailed error information

2.11.2. Log Files

System generates detailed logs with timestamps for debugging purposes. These files are generated on the logs_path directory specified in the system configuration file if requested. There is also detail operation logs of the PLC musol library

2.11.3. Support

For technical support or additional documentation, refer to:

- MUSOL Polarimeter library documentation
- SONG spectrograph documentation
- Application log files for detailed error information